Various Channel Models in the Current Codebase

Abstract

This document gives a methods-level description of all physical noise channels currently implemented in `code/quantum_simulation/noise_engine.py` and `code/quantum_simulation/noise_channels.p` For each channel, we provide: (i) mathematical construction, (ii) parameter meaning and selection guidance, and (iii) exact simulation semantics used by the code. The target usage is direct reuse in paper Methods/Setup sections.

Contents

1 Unified Simulation Setup **1**

2 Channel I: Google-Like Global Gate Depolarizing (`google_global`) **2**
2.1 Channel construction 2
2.2 Parameter selection 2

3 Channel II: Google-Like Gate-Specific Depolarizing (`google_gate_specific`) **3**
3.1 Channel construction 3
3.2 Parameter selection 3

4 Channel III: Idle Depolarizing Pauli Channel (`idle_depolarizing`) **3**
4.1 Channel construction 3
4.2 Parameter selection 4

5 Channel IV: Parametric Google Gate-Specific (`parametric_google`) **4**
5.1 Channel construction 4
5.2 Parameter selection 4

6 Channel V: Correlated Pauli Idle Channel (`correlated_pauli_noise_channel`) **4**
6.1 Design goal and constraints 4
6.2 Step 1: mismatch to target total probability 5
6.3 Step 2: convert total probability to per-axis mean 5
6.4 Step 3: Hidden-Markov telegraph process 5
6.5 Step 4: per-window sampling and injection 5
6.6 Physical interpretation and parameter selection 6
6.7 Statefulness and runtime semantics 6

7 Recommended Methods Reporting Template **6**

1 Unified Simulation Setup

We model the ideal circuit as an ordered instruction sequence. Let instruction index be $i = 1, \ldots, L$, and let ρ_i be the state before instruction i. The simulator applies

$$\rho_{i+1} = \mathcal{N}_i(\mathcal{U}_i(\rho_i)), \qquad \mathcal{U}_i(\rho) = U_i \rho U_i^\dagger, \tag{1}$$

where $\mathcal{N}_i$ is channel-specific and can depend on control mismatch, gate type, and idle-window position.

Timeline convention. The code builds a serial timeline with fixed idle gap τ_{idle} between adjacent instructions. Thus each circuit has $(L-1)$ idle windows:

$$W_i = [t_i^{\text{end}},\, t_i^{\text{end}} + \tau_{\text{idle}}], \qquad i = 1, \ldots, L-1. \tag{2}$$

Idle-only channels act on these windows, not on the final post-instruction gap.

Control mismatch notation. Let current control vector be $u \in \mathbb{R}^d$ and drifted optimum be $u^\star(t) \in \mathbb{R}^d$. Define global mismatch

$$m(t) = \frac{1}{d} \sum_{k=1}^{d} \left(u_k - u_k^\star(t)\right)^2 . \tag{3}$$

Probability clipping. Most channels use

$$\text{clip}_{p_{\max}}(x) = \min\{p_{\max}, \max\{0, x\}\}, \tag{4}$$

with $p_{\max} =$ `p_clip_max`.

2 Channel I: Google-Like Global Gate Depolarizing (`google_global`)

2.1 Channel construction

This channel adds depolarizing noise after each non-measure/non-reset gate. The same mismatch scalar $m(t)$ drives all gates at time t:

$$p_{1q}(t) = \text{clip}_{p_{\max}}\left(p_{1q}^0 + s_{1q}\, m(t)\right), \tag{5}$$

$$p_{2q}(t) = \text{clip}_{p_{\max}}\left(p_{2q}^0 + s_{2q}\, m(t)\right). \tag{6}$$

Here p_{1q}^0, p_{2q}^0 are base rates and s_{1q}, s_{2q} are sensitivity coefficients.

For a 1-qubit gate, code appends `DEPOLARIZE1(`p_{1q}`)`; for a 2-qubit gate, `DEPOLARIZE2(`p_{2q}`)`. Stim semantics are uniform Pauli mixtures:

$$\mathcal{D}_1(p)(\rho) = (1-p)\rho + \frac{p}{3} \sum_{P \in \{X,Y,Z\}} P\rho P, \tag{7}$$

$$\mathcal{D}_2(p)(\rho) = (1-p)\rho + \frac{p}{15} \sum_{P \in \mathcal{P}_2 \setminus \{II\}} P\rho P. \tag{8}$$

2.2 Parameter selection

A practical calibration path is:

G1. Fix p_{1q}^0, p_{2q}^0 from hardware baseline characterization.

G2. Estimate expected mismatch scale $\sigma_m \approx \mathbb{E}[m(t)]$ in the intended RL/control regime.

G3. Choose sensitivities from desired dynamic range:

$$s_{1q} \approx \frac{\Delta p_{1q}}{\sigma_m}, \qquad s_{2q} \approx \frac{\Delta p_{2q}}{\sigma_m}, \tag{9}$$

where Δp is the target increase from baseline at typical mismatch.

G4. Set $p_{\max}$ large enough not to clip normal operation, but low enough to avoid unphysical saturation.

3 Channel II: Google-Like Gate-Specific Depolarizing (google_gate_specifi

3.1 Channel construction

This model preserves the same depolarizing insertion rule as Channel I, but uses gate-specific mismatch features. Control is partitioned as

$$u = (u^{(1q)}, u^{(2q)}), \quad u^{(1q)} \in \mathbb{R}^{N_{1q}}, \quad u^{(2q)} \in \mathbb{R}^{N_{2q}}. \tag{10}$$

Each instruction is deterministically assigned a slot index (using stable hash of gate name + qubit targets):

$$j_{1q} = h_{1q}(\text{inst}), \qquad j_{2q} = h_{2q}(\text{inst}). \tag{11}$$

Per-instruction probabilities are

$$p_{1q,i} = \text{clip}_{p_{\max}}\left(p^0_{1q} + s_{1q}\left(u^{(1q)}_{j_{1q}} - u^{\star(1q)}_{j_{1q}}(t_i)\right)^2\right), \tag{12}$$

$$p_{2q,i} = \text{clip}_{p_{\max}}\left(p^0_{2q} + s_{2q}\left(u^{(2q)}_{j_{2q}} - u^{\star(2q)}_{j_{2q}}(t_i)\right)^2\right). \tag{13}$$

3.2 Parameter selection

S1. Choose N_{1q}, N_{2q} to control parameter sharing. Larger slot counts reduce hash collisions; smaller counts enforce sharing.

S2. Keep p^0_{1q}, p^0_{2q} at calibrated baselines, then tune s_{1q}, s_{2q} for desired control leverage.

S3. Verify collision statistics for the target circuit family, since collision structure changes effective model capacity.

Remark 1. *With one slot ($N_{1q} = N_{2q} = 1$), this channel degenerates toward global behavior. With many slots, it approximates gate-by-gate control sensitivity.*

4 Channel III: Idle Depolarizing Pauli Channel (idle_depolarizing)

4.1 Channel construction

This is an action-independent, idle-window-only Pauli channel. Given per-idle total probability p_{idle} and axis weights $w_x, w_y, w_z \geq 0$, define normalized weights

$$\tilde{w}_\alpha = \frac{w_\alpha}{w_x + w_y + w_z}, \qquad \alpha \in \{x, y, z\}. \tag{14}$$

Then per-idle axis probabilities are

$$p_x = \tilde{w}_x p_{\text{idle}}, \quad p_y = \tilde{w}_y p_{\text{idle}}, \quad p_z = \tilde{w}_z p_{\text{idle}}. \tag{15}$$

In code, this is implemented via constant rate functions $\lambda_\alpha = p_\alpha / \tau_{\text{idle}}$ integrated over each idle window, which exactly recovers Eq. (15).

At each idle window and each qubit, simulator appends `PAULI_CHANNEL_1([`p_x, p_y, p_z`])`.

4.2 Parameter selection

I1. Choose p_{idle} from target per-window error budget.

I2. Use equal weights for isotropic idle noise; use biased weights for dephasing-dominant or amplitude-like regimes.

I3. Ensure validity: $p_x, p_y, p_z \geq 0$ and $p_x + p_y + p_z \leq 1$.

I4. If starting from a per-circuit budget P_{circ} over N_w idle windows, convert by

$$p_{\text{idle}} = 1 - (1 - P_{\text{circ}})^{1/N_w}. \tag{16}$$

5 Channel IV: Parametric Google Gate-Specific (`parametric_google`)

5.1 Channel construction

This channel uses the same gate-specific machinery as Channel II, but introduces explicit regime parameters $a =$ `channel_regime_a` and $b =$ `channel_regime_b`. Code rescales both base rates and sensitivities:

$$p^0_{1q} \mapsto a\, p^0_{1q}, \qquad s_{1q} \mapsto a\, s_{1q}, \tag{17}$$

$$p^0_{2q} \mapsto b\, p^0_{2q}, \qquad s_{2q} \mapsto b\, s_{2q}. \tag{18}$$

Hence Eq. (13) becomes

$$p_{1q,i} = \text{clip}_{p_{\max}}\left(a\left[p^0_{1q} + s_{1q}\delta^2_{1q,i}\right]\right), \tag{19}$$

$$p_{2q,i} = \text{clip}_{p_{\max}}\left(b\left[p^0_{2q} + s_{2q}\delta^2_{2q,i}\right]\right), \tag{20}$$

where $\delta_{1q,i}, \delta_{2q,i}$ denote corresponding slot mismatch magnitudes.

5.2 Parameter selection

P1. Use (a, b) for structured regime sweeps while keeping architecture fixed.

P2. a primarily controls 1q channel severity; b controls 2q severity.

P3. For fair comparisons, keep all non-channel hyperparameters fixed and vary only (a, b).

6 Channel V: Correlated Pauli Idle Channel (`correlated_pauli_noise_chann`

6.1 Design goal and constraints

This is the only stateful, temporally correlated channel in current code. It is built as a Hidden-Markov Pauli process injected on idle windows. The implementation enforces:

C1. Qubit independence: each qubit has its own hidden chain(s).

C2. Direction independence: X/Y/Z use identical (f, g) statistics.

C3. Equal axis weights are required ($w_x = w_y = w_z$).

6.2 Step 1: mismatch to target total probability

Using global mismatch at current step,

$$m = \frac{1}{d}\sum_{k=1}^{d}(u_k - u_k^\star(0))^2, \tag{21}$$

define base strength

$$s_{\text{base}} = p_{1q}^0 + s_{1q}m, \tag{22}$$

and raw target

$$p_{\text{target}} = \text{clip}_{p_{\max}}(g\, s_{\text{base}}). \tag{23}$$

Two g modes:

- `per_window`: $p_{\text{nom}} = p_{\text{target}}$.
- `per_circuit`: for normalization windows N_w,

$$p_{\text{nom}} = 1 - (1 - p_{\text{target}})^{1/N_w}. \tag{24}$$

Then clip p_{nom} to $[0, p_{\max}]$.

6.3 Step 2: convert total probability to per-axis mean

Let p be per-axis Bernoulli mean (same for X/Y/Z). The non-identity probability in one idle window is

$$p_{\text{tot}} = 1 - \Pr(I) = 1 - \left((1-p)^3 + p^3\right) = 3p(1-p). \tag{25}$$

Code solves this as

$$p = \frac{1 - \sqrt{1 - \frac{4}{3}p_{\text{tot}}}}{2}, \qquad p_{\text{tot}} \le 0.75. \tag{26}$$

Hence it clips $p_{\text{tot}} = \min\{p_{\text{nom}}, 0.75 - \varepsilon\}$.

6.4 Step 3: Hidden-Markov telegraph process

For each qubit q and each axis $\alpha \in \{X, Y, Z\}$, introduce hidden state $s_{\alpha,q,t} \in \{0, 1\}$ with emission probs

$$p_{\text{low}} = 0, \qquad p_{\text{high}} = \min\{2p, 1\}. \tag{27}$$

Transition matrix is symmetric:

$$T = \begin{pmatrix} 1-\gamma & \gamma \\ \gamma & 1-\gamma \end{pmatrix}, \qquad \gamma = \frac{1 - e^{-f\tau_{\text{idle}}}}{2}, \tag{28}$$

where $f =$ `corr_frequency_hz`. Initial distribution is $(0.5, 0.5)$.

6.5 Step 4: per-window sampling and injection

For each idle window t:

H1. Sample events $e_{\alpha,q,t} \sim \text{Bernoulli}(p_{s_{\alpha,q,t}})$.

H2. Advance hidden states with Eq. (28).

H3. Append Pauli gates to noisy circuit: `X` on all qubits with $e_{X,q,t} = 1$, similarly for `Y`, `Z`.

Multiple axes may fire on one qubit in the same window.

6.6 Physical interpretation and parameter selection

Correlation frequency f. From Eq. (28), one-step correlation factor is $\rho = e^{-f\tau_{\text{idle}}}$. Thus physical correlation time is approximately

$$\tau_c \approx \frac{1}{f}. \tag{29}$$

Small f gives longer memory (low-frequency drift), large f gives faster mixing.

Strength g. From Eq. (23), g scales the mismatch-driven base strength. For a target budget P and estimated mismatch m, a first-order choice is

$$g \approx \frac{P}{p_{1q}^0 + s_{1q}m} \quad \text{(before clipping effects).} \tag{30}$$

g-mode choice.

- Use `per_window` when all compared experiments have similar circuit length.
- Use `per_circuit` when comparing different round/circuit lengths and you want length-normalized budgets.

6.7 Statefulness and runtime semantics

This model is stateful across consecutive `apply(...)` calls within one shot. Therefore:

R1. apply-result cache is disabled by design;

R2. hidden states are reset at shot start via `start_shot()`;

R3. Steane adapter forces effective `shot_workers=1` for this channel.

These constraints preserve correct temporal-memory semantics.

7 Recommended Methods Reporting Template

For reproducibility, report at least:

1. Exact channel key and resolved mode (avoid only writing "auto").
2. Base rates/sensitivities: $p_{1q}^0, p_{2q}^0, s_{1q}, s_{2q}, p_{\max}$.
3. For gate-specific models: N_{1q}, N_{2q} and slot-mapping policy.
4. For idle models: τ_{idle}, axis weights, and per-window or per-circuit budget definition.
5. For correlated model: (f, g), g-mode, normalization windows N_w, and any clipping regime.
6. Runtime semantics affecting statistics: trace mode, shot workers, state reset policy.